

# Linux, Networking & Git Practice Assignment

Racheal Kuranchie

## Part 1 Linux Practice Log

*pwd*

```
/Users/rachealkuranchie
```

*ls -la*

```
total 650904
drwxr-x---+ 140 rachealkuranchie  staff      4480 Jul 10 11:34 .
drwxr-xr-x   9 root               admin       288 Jun 22 02:25 ..
-r-----   1 rachealkuranchie  staff         7 Sep 30  2023 .CFUserTextEncoding
-rw-r--r--@  1 rachealkuranchie  staff    12292 Jul  6 20:14 .DS_Store
drwx-----+ 338 rachealkuranchie  staff    10816 Jul  6 20:37 .Trash
drwxr-xr-x   4 rachealkuranchie  staff       128 Sep 10  2024 .anaconda
```

*mkdir week1-practice && cd week1-practice*

```
(base) 🐉 base rachealkuranchie@users-MacBook-Pro ~/Documents/week1-practice
```

*touch notes.txt*

```
(no output – file created)
```

*echo "hello" > notes.txt*

```
(no output – text written to file)
```

*cat notes.txt*

```
$ cat note.txt
cat: note.txt: No such file or directory

$ cat notes.txt
hello
```

*ls -la (before chmod)*

```
total 8
drwxr-xr-x   3 rachealkuranchie  staff       96 Jul 10 11:35 .
drwx-----@ 60 rachealkuranchie  staff    1920 Jul 10 11:34 ..
-rw-r--r--   1 rachealkuranchie  staff        6 Jul 10 11:35 notes.txt
```

*chmod 600 notes.txt*

```
(no output – permissions changed)
```

*ls -l notes.txt (permission change confirmed)*

```
total 8
drwxr-xr-x  3 rachealkuranchie  staff    96 Jul 10 11:35 .
drwx-----@ 60 rachealkuranchie  staff  1920 Jul 10 11:34 ..
-rw-----  1 rachealkuranchie  staff    6 Jul 10 11:35 notes.txt
```

## healthcheck.sh

```
#!/bin/bash
echo "checking ....."
curl google.com
echo "UP"
echo "DOWN"
```

*./healthcheck.sh*

```
checking .....
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
UP
DOWN
```

## Part 2 Networking Investigation

Site tested: github.com

*ping github.com*

```
PING github.com (140.82.121.3): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
Request timeout for icmp_seq 4
Request timeout for icmp_seq 5
Request timeout for icmp_seq 6
Request timeout for icmp_seq 7
Request timeout for icmp_seq 8
Request timeout for icmp_seq 9
Request timeout for icmp_seq 10
Request timeout for icmp_seq 11
Request timeout for icmp_seq 12
```

```
Request timeout for icmp_seq 13
Request timeout for icmp_seq 14
Request timeout for icmp_seq 15
Request timeout for icmp_seq 16
Request timeout for icmp_seq 17
^C
--- github.com ping statistics ---
19 packets transmitted, 0 packets received, 100.0% packet loss
```

*curl -I https://github.com*

```
HTTP/2 200
date: Fri, 10 Jul 2026 11:50:16 GMT
content-type: text/html; charset=utf-8
vary: X-PJAX, X-PJAX-Container, Turbo-Visit, Turbo-Frame, X-Requested-With,
      Accept-Language, Sec-Fetch-Site, Accept-Encoding, Accept, X-Requested-With
content-language: en-US
etag: W/"3fab0f2e45262c6f4950ae21810a4895"
cache-control: max-age=0, private, must-revalidate
strict-transport-security: max-age=31536000; includeSubdomains; preload
x-frame-options: deny
```

*dig github.com*

```
; <<>> DiG 9.10.6 <<>> github.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 42864
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;github.com.                IN  A

;; ANSWER SECTION:
github.com.                 27  IN  A    140.82.121.4
```

## Explanation

**Dig** command checks the DNS resolver layer; it checks what IP address a domain name points to and the DNS working.

**Ping** on the other hand, checks the network layer; it checks if the host can send a packet to the receiver and get a reply.

**Curl** checks the application layer to see whether you get an established connection to complete a handshake and get a valid http response.

**Traceroute** checks exactly where in the connection is dying.

**Lsof** checks what specific files and ports are currently in use and by what process

## Part 3 Git & Github

<https://github.com/Racheal777/platform-engineering>

## Part 4 Blameless Postmortem

Using the 2:14am checkout outage scenario.

### What Happened

At 2:14 am, an alert fired indicating that approximately 30% of customer checkout requests were failing. Fatima, the on-call engineer for the platform that week, began investigating immediately.

She started with the monitoring dashboard, which showed the error rate had spiked following a deployment made six hours earlier. Checking Git confirmed the exact commit and deploy time. She then SSHed into the affected server, reviewed the application logs, and grep-ed for the specific error signature. The logs pointed to a failure connecting to the payment provider's API.

To confirm, she ran curl against the payment provider's health endpoint directly from the server; it timed out. Working through a mental checklist of DNS, firewall rules, and provider-side issues, she traced the problem back to the six-hour-old deployment.

### Root Cause

The deployment had changed a configuration value that pointed the application to the wrong payment API region. This was not a code problem but a networking/configuration misconfiguration introduced during deployment.

The underlying issue was that this config value was not validated before or during deployment, so an incorrect region setting was able to reach production without being caught. Because the failure only affected payment connectivity rather than the whole application, it also went undetected for roughly six hours until the error rate crossed the alerting threshold, rather than being flagged immediately at deploy time.

### Resolution

Rather than attempt to debug the issue live in production, Fatima reverted the change using ``git revert`` and redeployed the last known-good version. The error rate dropped to zero within four minutes of the redeploy.

### Prevention

Concrete change: Add an automated validation step in the deployment pipeline that checks payment API region config values against a known allowlist before the deploy is allowed to proceed. If the config doesn't match an approved value, the deploy fails automatically rather than reaching production.

This would catch this exact class of error at deploy time, in seconds, instead of relying on customer-facing error rates to show the problem six hours later.